

Fixing Mutual Authentication

Encryption does not ensure authenticity: use MAC to authenticate messages

Prevent redirection: identify party in challenge

Better to use separate keys for each direction

Prevent replay and reorder

- Identify flow and connection
- Prevent use of old challenge: randomness, time, or state

Do not provide the adversary with oracle access! (like in SNA)

- Do not compute values from Adversary
- Include self-chosen nonce in the protected reply

Secure Two-Party Handshake Protocol (2PP)

Use MAC rather than encryption to auth

Prevent redirection: include identities (A, B)

Prevent replay and reorder:

- Nonces (N_A, N_B)
- Separate 2nd and 3rd flows: 3 vs 2 input blocks

Provably secure, since MACs are deterministic

New nonces are generated for each flow, prevents concurrent attack

Flow numbers ensure prevention of redirection

Authenticated Request-Control Protocols

Besides authenticating entities, these protocols auth the exchange of a request and a response between the entities.

Required properties:

- Request Authentication: the request was indeed sent by the sender.
- Response Auth: the response was indeed sent by the receiver (to which the request was intended for)
- No Replay

Five variants:

- 2PP-RR (two party protocol, request-response)
- 2RT-2PP (2 round trip) (a round trip is two flows)
- Counter-based-RR
- Time-based RR
- Key-exchange (session key)

2PP-RR

A three-flow nonce-based protocol.

Significant drawback:

- The request is sent by the responder and the initiator sends the response.
- So, the initiator must wait for a request rather than sending it

2RT-PP

A four-flow nonce-based protocol.

Mainly fixes the drawback of 2PP-RR

Counter-Based Authenticated RR

Simple stateful (counter) solution, requiring only one round:

- Unidirectional (if bidirectional is needed, a separate instance of the protocol for each direction needs to be executed)
- Parties maintain synchronized counter i of requests (and responses) to avoid replay attacks
- Recipient validates if counter received is $i + 1$
- Both parties must remember the counter

Counters are only initialized once

Time-Based Auth RR

Simple stateful (time) solution, requiring only one round:

- Use local clocks T_A, T_B instead of counters with two assumptions: bounded delays and bounded clock skews
- Responder (Bob):
 - Rejects request if $T_B > T_A + \Delta$ OR if he received a larger T_A already (where $\Delta = \Delta_{skew} + \Delta_{delay}$)
 - Maintains last T_A received until $T_A + \Delta$
- Initiator does not need any state, when can receiver discard this?

Don't reuse keys across parties (TWO party protocol)

2RT-2PP with Confidentiality

Secure connection: authentication, freshness, and secrecy

- Independent keys: for encryption $k.e$, for auth $k.a$
- How can we derive them both from a single key k ? The PRF from before: $k.e = PRF_k("Encrypt")$, $k.a = PRF_k("MAC")$

Improve security by changing keys between sessions?

2PP Key Exchange Protocol

Allows generating independent session keys (i.e. each session will have its own key denoted as k_i^S)

These keys are derived from the shared key that we now call a long-term 'master key' denoted as k^M

Improves security:

- Exposure of a session key does not expose the master key, or keys of other sessions
- Thus, limited amount of ciphertext is exposed if a session key is exposed (only that session, rather than all the ciphertexts of other sessions)

$A, N_{A.i}$

$N_{A.i}, N_{B.i}, PRF_{k_M}(2 || 'A \leftarrow B || N_{A.i} || N_{B.i})$

$N_{B.i}, PRF_{k_M}(3 || 'A \rightarrow B' || N_{A.i} || N_{B.i})$

Session key: $k_i^S = PRF_{k_M}(N_{A.i} || N_{B.i})$

Use session key to generate MAC and Encryption key

A secure PRF is also a secure MAC, this example uses only one cryptographic primitive (the PRF, for session key and handshake).

You can't use the same key against two different crypto primitives

Key Distribution Center (KDCs): Establish a shared key b/w two or more entities, usually with the help of a trusted 3rd party reference

KDC

Has huge trust assumption based on centralization

Will focus on three party protocols: Alice, Bob, and KDC

KDC: shares long-term master keys with all parties (each party will have a key for encryption and another for MAC)

- Denoted as $k_A^M, k_B^M, \dots$ for MAC and $k_A^E, k_B^E, \dots$ for encryption

Goal: help parties (A, B) to establish a shared master key k_{AB} , based on which the parties generate two keys for MAC and encryption, namely k_{AB}^M and k_{AB}^E

We will study simplified versions of Kerberos (secure) and GSM (insecure)

The Kerberos KDC Protocol

KDC Shares keys k_A^E (enc), k_A^M (MAC) with Alice, and k_B^E, k_B^M with Bob

Goal: Alice and Bob share k_{AB}^M , then derive k_{AB}^E, k_{AB}^M

KDC performs access control as well, controlling with whom Alice can contact

Uses timestamp for freshness

If an attacker compromises the KDC, they have control over the master key and can therefore decrypt any message

The GSM Handshake Protocol

Mobile client, identified by i (IMSI: International Mobile Subscriber Identifier)

Visited network (aka Base station), not fully trusted

Home network: trusted, shares key k_i with client i

A5 function used to generate OTP is not a PRF (not secure)

Attacks on GSM Handshake Protocol

Visited network impersonation replay attack

Ciphersuite downgrade attack

Visited-Network Impersonation Attack

Cryptanalysis phase: attacker exposes K_c based on the cyphertexts it collected in the eavesdropping phase

In the impersonate phase, the attacker will send the same r and s from before (replay attack), which will lead to the same K_c it obtained in the cryptanalysis phase

GSM is using weak primitives that can be broken (OWF but not PRF A5/1, A5/2)

GSM doesn't check for freshness with r or session key.

GSM Ciphersuite Downgrade Attack

Can be used to setup Visited-Network attack

A ciphersuite is the set of cryptographic schemes used in protocol execution

Ciphersuite negotiation:

- Mobile sends a list of cipher-suites it supports
- Visited network selected the strongest/most secure one that it also supports

- Goal of negotiation is to support interoperability b/w devices of different capabilities

A MitM attacker may trick these parties to use a weak suite (even if they can support a stronger one)

Works due to key reuse in GSM (same key is used across various encryption schemes)

Improving Resiliency to Key Exposure

Forward Secrecy

So far: session key $k_i^S \not\Rightarrow k_j^S$ (expose no other keys), and master key was fixed for all sessions.

Idea; Change the master key each session: $k_1^M, k_2^M, \dots$

Forward Secrecy (FS): master key $k_i^M \not\Rightarrow k_j (j < i)$

I.e., k_i^M (and k_i^S) don't expose keys, communication of previous sessions ($j < i$)

Gives secrecy to the past (but not the future)

Forward Secrecy 2PP Key Exchange

This protocol generates a different master key for each session i denoted as k_i^M

The initial master key shared between the parties is k_0^M